

Session 8 — Smart Contract Security

Buổi 8 — An toàn hợp đồng thông minh

Exploit, then patch / Khai thác rồi vá

Blockchain Technology and Cryptocurrency

TS. Nguyễn Trung Thành

Reentrancy • The DAO • access control • oracles • the secure-dev lifecycle • Slither

Objectives / Mục tiêu

1. Recognize the major **vulnerability classes**: reentrancy, access control, arithmetic, oracle, replay, DoS, randomness
Nhận diện các lớp lỗ hổng chính
2. Explain **reentrancy** end-to-end and defend with **Checks-Effects-Interactions + guard + pull-over-push**
Giải thích reentrancy và phòng thủ bằng CEI + guard + pull-over-push
3. Run the **secure-development lifecycle**: threat model → static analysis (**Slither**) → fuzz/invariant → audit → bug bounty → monitor
Chạy vòng đời phát triển an toàn
4. In the lab: **exploit** a vulnerable bank, then **patch** it so the same attack fails
Trong lab: khai thác một ngân hàng lỗi rồi vá để cùng cuộc tấn công thất bại
5. Prepare for the **midterm MCQ next session (S9)**
Chuẩn bị cho bài trắc nghiệm giữa kỳ buổi sau

Agenda — 3 hours / 3 giờ

Time	Content / Nội dung
0:00–0:55	Vulnerability classes — các lớp lỗ hổng
0:55–1:05	Break / <i>giải lao</i>
1:05–2:00	Secure-dev lifecycle + case studies — quy trình an toàn & tình huống
2:00–2:10	Break / <i>giải lao</i>
2:10–3:05	Lab 8 : exploit → patch → Slither + midterm preview

Recap / Ôn lại (Sessions 6-7)

- You can **write, test and deploy** contracts with Solidity + Hardhat (S6)
Bạn đã viết, kiểm thử và triển khai hợp đồng với Solidity + Hardhat
- You built **token standards** — ERC-20/721/1155 — value that moves by code (S7)
Bạn đã dựng các chuẩn token — giá trị chuyển động bằng mã
- **Immutable + holds money + public = adversarial.** Anyone on Earth can call any function.
Bất biến + giữ tiền + công khai = môi trường đối kháng. Ai cũng gọi được mọi hàm.

Today: how that code gets attacked — and how you stop it.

Hôm nay: mã đó bị tấn công thế nào — và bạn chặn ra sao.

Why smart-contract security is different / Vì sao khác biệt

- **No patch in place** — a deployed bug is permanent until you migrate funds out
Không vá tại chỗ — lỗi đã deploy là vĩnh viễn cho tới khi di dời tiền
- **The code is the money** — a bug is not a crash, it is a direct theft
Mã chính là tiền — lỗi không phải "treo máy" mà là mất cắp trực tiếp
- **Public + composable** — anyone calls you; other contracts call you mid-transaction
Công khai + kết hợp được — người khác và hợp đồng khác gọi bạn giữa giao dịch
- **Adversary is funded & patient** — flash loans give attackers millions for one block
Kẻ tấn công có vốn & kiên nhẫn — flash loan cho họ hàng triệu \$ trong một block

Since 2020, on-chain hacks have drained **billions of USD** cumulatively. Most are *preventable* bug classes we cover today. / Từ 2020, các vụ hack đã cuốn đi **hàng tỷ USD** — phần lớn thuộc các lớp lỗi phòng tránh được.

Hour 1 / Giờ 1

Vulnerability classes / Các lớp lỗ hổng

Reentrancy first — the one that made history

The attacker's mindset / Tư duy kẻ tấn công

For every function, ask / Với mỗi hàm, hãy hỏi:

- **Who may call this?** Is the check correct, or missing? *Ai được gọi? Kiểm tra đúng hay thiếu?*
- **What if it is called again before it finishes?** (reentrancy) *Nếu bị gọi lại trước khi xong?*
- **What if inputs are extreme?** 0, max, self, empty. *Đầu vào cực đoan thì sao?*
- **What do I trust?** External calls, oracles, `delegatecall` targets. *Tôi tin cái gì?*
- **Can I make it fail for everyone else?** (DoS) *Có thể làm nó hỏng cho mọi người?*

Security review = adversarial reading. Assume every external address is the attacker. / Rà soát an toàn = đọc theo lối đối kháng. Coi mọi địa chỉ ngoài là kẻ tấn công.

Reentrancy — the mechanism / Cơ chế reentrancy

Reentrancy: rút cạn ngân hàng vs bản vá CEI — draining the bank vs the CEI fix

Trái: interaction TRƯỚC effect ⇒ tái nhập rút nhiều lần • Phải: effect TRƯỚC interaction + guard ⇒ revert • Left: interaction before effect drains it; right: effect-first + guard reverts

DỄ TỔN THƯƠNG — VULNERABLE (interactions before effects)

1 • attacker.attack()

gửi 1 ETH stake, gọi bank.withdraw() / deposit 1 ETH, call withdraw()

2 • withdraw() lần 1 — CHECK

require(balances[atk] > 0) ✓

3 • INTERACTION (LỖI / the bug)

msg.sender.call{value: bal}("") → attacker.receive()

4 • receive(): TÁI NHẬP withdraw() lần 2 / RE-ENTER

balances[atk] CHƯA bị gán 0 — vẫn > 0! / not zeroed yet

5 • CHECK qua LẦN NỮA → trả tiền lại

lặp cho tới khi ngân hàng cạn / loops until the bank is empty

6 • balances[atk] = 0

EFFECT chạy sau cùng — quá muộn / effect runs last, too late

Kết quả: kẻ tấn công rút gấp N lần số dư ⇒ cuốn sạch tiền gửi của mọi người. Vụ The DAO (2016) mất ~3,6 triệu ETH theo đúng cơ chế này và dẫn tới cú hard-fork ETH/ETC. The attacker withdraws N× its balance, draining every depositor. The DAO (2016) lost ~3.6M ETH this exact way, forcing the ETH/ETC fork.

ĐÃ VÁ — PATCHED (Checks → Effects → Interactions + guard)

1 • attacker.attack()

gửi 1 ETH stake, gọi withdraw() / same attacker contract

2 • nonReentrant: khóa = ENTERED

modifier đặt cờ chống tái nhập / guard sets the lock

3 • CHECK: require(bal > 0) ✓

4 • EFFECT: balances[atk] = 0

cập nhật trạng thái TRƯỚC mọi lời gọi ngoài / state settled BEFORE any call

5 • INTERACTION: call{value: bal} → receive()

pull-over-push cũng áp dụng ở đây / pull-over-push also applies

6 • receive() tái nhập withdraw() → REVERT

guard đã khóa / balances đã = 0 ⇒ chặn đứng / guard locked or bal already 0

Kết quả: kẻ tấn công chỉ lấy lại đúng 1 ETH stake của mình; tiền gửi an toàn. Ba tầng phòng thủ độc lập: (1) CEI, (2) ReentrancyGuard, (3) pull-over-push. The attacker recovers only its own 1 ETH stake — deposits are safe. Three independent defenses: CEI, a reentrancy guard, and pull-over-push withdrawals.

The vulnerable withdraw / Hàm rút bị lỗi

```
function withdraw() external {
    uint256 bal = balances[msg.sender];
    require(bal > 0, "nothing to withdraw");
    (bool ok,) = msg.sender.call{value: bal}(""); // INTERACTION first (BUG)
    require(ok, "transfer failed");
    balances[msg.sender] = 0; // EFFECT last (too late)
}
```

- `.call{value:}` hands control to `msg.sender`'s `receive()` **before** the balance is zeroed
`.call` trao quyền cho `receive()` của kẻ gọi TRƯỚC khi số dư về 0
- The attacker's `receive()` calls `withdraw()` again — the check still passes
`receive()` gọi lại `withdraw()` — điều kiện vẫn đúng
- Loop until the bank is empty. The attacker withdraws **Nx** their balance.
Lặp cho tới khi cạn. Rút gấp N lần số dư.

Case study: The DAO, 2016 / Tình huống: The DAO

- **The DAO**: a crowd-funded investment fund on Ethereum; raised ~**12.7M ETH** ($\approx$ \$150M then)
Quỹ đầu tư gọi vốn cộng đồng; huy động ~12,7 triệu ETH
- June 2016: attacker used **exactly this reentrancy** in the `splitDAO` withdrawal path
Kẻ tấn công dùng đúng reentrancy này trong đường rút tiền
- Drained ~**3.6M ETH** before being stopped by the fund's own structure
Rút ~3,6 triệu ETH trước khi bị chặn
- Response split the community: a **hard fork** reversed it → today's **ETH**; the un-forked chain lives on as **Ethereum Classic (ETC)**
Phản ứng chia rẽ cộng đồng: một hard fork đảo ngược ⇒ ETH ngày nay; chuỗi không fork là ETC

"Code is law" met its hardest test. The bug was one line of ordering. / "Mã là luật" gặp phép thử khó nhất — lỗi chỉ là một dòng sai thứ tự.

Reentrancy is not just one shape / Reentrancy có nhiều dạng

Variant / Biến thể	Idea / Ý tưởng
Single-function	Re-enter the <i>same</i> function (the DAO) / tái nhập cùng hàm
Cross-function	Re-enter a <i>different</i> function sharing state / hàm khác dùng chung trạng thái
Cross-contract	Re-enter via a <i>third</i> contract holding shared state / qua hợp đồng thứ ba
Read-only	A <code>view</code> returns stale state mid-call; a reader acts on it / <code>view</code> trả trạng thái cũ giữa chừng

This is why **CEI + a guard** matter together: CEI stops single-function, the guard and careful state layout stop the cross-* variants. / Vì thế CEI + guard bổ trợ nhau.

Defense 1 — Checks-Effects-Interactions / Phòng thủ 1: CEI

Mẫu Checks-Effects-Interactions — the CEI pattern

Thứ tự BẮT BUỘC trong mọi hàm chạm tới tài sản / trạng thái • The mandatory ordering in any function that touches value or state



Defense 2 & 3 — guard + pull-over-push / Guard & pull-over-push

```
function withdraw() external nonReentrant {    // guard: locks re-entry
    uint256 bal = balances[msg.sender];
    require(bal > 0);
    balances[msg.sender] = 0;                // EFFECT before INTERACTION
    (bool ok,) = msg.sender.call{value: bal}("");
    require(ok, "transfer failed");
}
```

- **nonReentrant** (OpenZeppelin `ReentrancyGuard`): a status flag; a nested call reverts *cờ trạng thái; lời gọi lồng nhau bị revert*
- **Pull-over-push**: let each user *withdraw their own* funds; never loop paying many addresses
cho từng người TỰ rút; đừng đẩy tiền hàng loạt
- **Defense in depth**: use all three — do not rely on one line being correct forever
phòng thủ nhiều lớp: dùng cả ba

Access control — the #1 loss category / Kiểm soát truy cập

- **Missing modifier:** a sensitive function has no `onlyOwner` / `onlyRole`
Thiếu modifier trên hàm nhạy cảm
- **Wrong check:** `onlyOwner` on the getter but not the setter; role never granted
Kiểm tra sai chỗ
- **Unprotected initializer** (proxy pattern, hook S7): anyone calls `initialize()` and becomes owner
Hàm khởi tạo không bảo vệ — ai cũng gọi `initialize()` để thành owner

```
function setOwner(address n) external { owner = n; } // anyone!  
function setOwner(address n) external onlyOwner { owner = n; } // fixed
```

Always: least privilege, explicit modifiers, events on privileged actions, two-step ownership transfer. / Nguyên tắc: đặc quyền tối thiểu, modifier rõ ràng, phát event, chuyển quyền 2 bước.

tx.origin vs msg.sender / Bẫy phishing

```
require(tx.origin == owner); // WRONG – phishable  
require(msg.sender == owner); // RIGHT
```

- `tx.origin` = the **EOA that started the tx**; `msg.sender` = the **immediate caller**
tx.origin = ví gốc khởi tạo giao dịch; *msg.sender* = người gọi trực tiếp
- Attack: owner is tricked into calling `Evil.claim()`. Inside, `Evil` calls your contract.
Owner bị lừa gọi Evil.claim(); bên trong, Evil gọi hợp đồng của bạn.
- Now `tx.origin == owner` is **true** → Evil passes your check and drains you
tx.origin == owner là ĐÚNG ⇒ Evil qua được kiểm tra

Never use `tx.origin` for authorization. / Không bao giờ dùng `tx.origin` để phân quyền.

Case study: Parity multisig, 2017 / Tình huống: Parity

- Parity's multisig wallets shared a single **library contract** via `delegatecall`
Ví đa chữ ký Parity dùng chung một hợp đồng thư viện qua `delegatecall`
- The library's `initWallet` was **left unprotected**; a user called it and became owner of the library...
Hàm khởi tạo thư viện không được bảo vệ; một người gọi nó và thành owner
- ...then called `selfdestruct`. The library vanished; every wallet depending on it **froze**
...rồi gọi `selfdestruct`. Thư viện biến mất; mọi ví phụ thuộc bị đóng băng
- ~**514k ETH** permanently frozen (not stolen — **stuck forever**)
~514 nghìn ETH bị khóa vĩnh viễn — không bị trộm mà kẹt mãi

Unprotected initializer + `delegatecall` + `selfdestruct` — three lessons in one incident. / Ba bài học trong một sự cố.

Arithmetic — overflow & underflow / Số học

Era / Thời kỳ	Behavior / Hành vi
Solidity < 0.8	<code>uint8 x = 255; x + 1 == 0</code> — silent wrap; needed SafeMath
Solidity ≥ 0.8	overflow/underflow reverts automatically — no SafeMath needed
<code>unchecked { }</code>	opts <i>back out</i> of checks for gas — a modern footgun

```
unchecked { balance -= amount; } // if amount > balance -> underflow, NO revert
```

- Also watch **precision/rounding**: integer division truncates; order operations to lose least

Cẩn thận làm tròn: chia số nguyên bị cắt; sắp thứ tự phép tính để mất ít nhất

- Multiply **before** divide; scale by 1e18; never assume a token has 18 decimals

Nhân trước, chia sau; nhân hệ số 1e18; đừng giả định 18 chữ số thập phân

Oracle & price manipulation / Thao túng oracle & giá

- Many DeFi bugs are **not** in your Solidity — they trust a **manipulable price**
Nhiều lỗi DeFi không nằm trong Solidity của bạn — mà tin vào một mức giá bị thao túng
- **Spot price** from a single DEX pool = whatever the last trade set it to
Giá "spot" từ một pool DEX = giá của giao dịch cuối
- **Flash loans** (hook S10) let an attacker borrow millions with no collateral **for one tx**, skew the pool, read the bad price, profit, repay — all atomically
Flash loan cho vay hàng triệu \$ trong một giao dịch để bẻ cong giá

Defenses / Phòng thủ: time-weighted average price (**TWAP**), decentralized oracles (**Chainlink**), multiple independent sources, sanity bounds. / Dùng TWAP, oracle phi tập trung (Chainlink), nhiều nguồn, chặn biên hợp lý.

Front-running & MEV / Chạy trước & MEV

- The mempool is **public**: your pending tx is visible before it is mined
Mempool công khai: giao dịch chờ của bạn bị thấy trước khi được khai thác
- Searchers reorder/insert txs for profit — **MEV** (maximal extractable value), recall S5
Người tìm kiếm sắp lại/chèn giao dịch để trục lợi — MEV (nhắc lại S5)
- **Sandwich attack**: buy before your swap, sell after — you get the worst price
Tấn công "kẹp bánh": mua trước, bán sau giao dịch của bạn

Defenses / Phòng thủ: slippage limits, **commit-reveal**, private mempools/relays, deadline params. / Giới hạn trượt giá, commit-reveal, mempool riêng, tham số hạn chót.

More classes to know / Các lớp khác cần biết (1/2)

Bug / Lỗi	One-line risk / Rủi ro
Unchecked external-call return	<code>.call</code> / <code>.send</code> returns <code>false</code> , you ignore it → state diverges / bỏ qua giá trị trả về
<code>delegatecall</code> to untrusted	callee runs in your storage context → total takeover / chạy trong storage của bạn
Signature replay (hook S3)	reuse a valid signature; fix: nonce + chainId + domain (EIP-712) / phát lại chữ ký

`delegatecall` runs foreign code with your storage, `msg.sender`, and balance. Trust the target absolutely or not at all. / `delegatecall` chạy mã lạ với storage của bạn — tin tuyệt đối hoặc không dùng.

More classes to know / Các lớp khác cần biết (2/2)

Bug / Lỗi	One-line risk / Rủi ro
DoS: unbounded loop	a growing array makes a function exceed the block gas limit / vòng lặp không giới hạn
DoS: gas griefing	one reverting recipient blocks a payout loop → pull-over-push / một người revert chặn cả vòng
Weak randomness	<code>block.timestamp</code> / <code>blockhash</code> are proposer-influenced → VRF / commit-reveal
<code>selfdestruct</code> / force-feed	ETH can be <i>forced</i> in; never rely on <code>address(this).balance</code> for logic / có thể bị ép nhận ETH

Standard vocab: **SWC Registry** (Smart Contract Weakness Classification) and **SCSVS** (verification standard). / Bộ từ vựng chuẩn: SWC Registry và SCSVS.

Break 10' / Giải lao

Next: how professionals ship secure contracts
Tiếp theo: cách chuyên gia phát hành hợp đồng an toàn

Hour 2 / Giờ 2

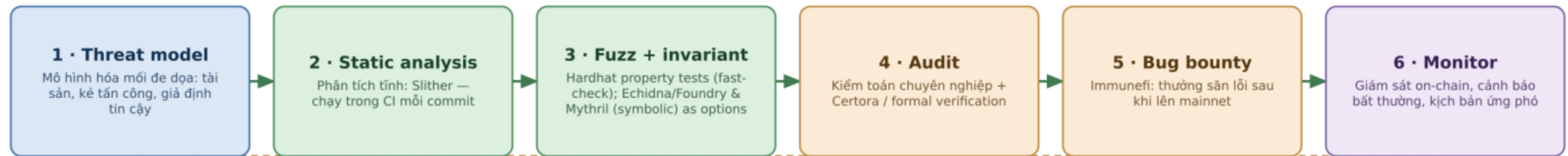
The secure-development lifecycle / Vòng đời phát triển an toàn

No single stage is a silver bullet / Không khâu nào là "viên đạn bạc"

The lifecycle / Vòng đời

Vòng đời phát triển an toàn — the secure development lifecycle

Không một khâu nào là 'viên đạn bạc' — các lớp bù trừ cho nhau • No single stage is a silver bullet — the layers compensate



Công cụ & chuẩn kiểm / tooling & checklists

Slither, Hardhat property tests, Echidna, Mythril, Certora • SWC Registry, SCSVS, Solidity security-considerations, ToB building-secure-contracts

"Audited ≠ safe" — đã kiểm toán KHÔNG có nghĩa là an toàn

Kiểm toán là ảnh chụp tại một thời điểm, phạm vi giới hạn. Cầu nối (bridge) vẫn thống trị thiệt hại: Ronin ~625 triệu \$ (2022), Wormhole ~325 triệu \$. An audit is a scoped snapshot; bridge hacks still dominate losses.

Đọc chuỗi trái→phải là thứ tự thời gian, nhưng bảo mật là VÒNG LẶP: mỗi phát hiện quay lại đầu nguồn. Chi phí sửa một lỗi hỏng tăng vọt từ trái sang phải — rẻ nhất khi bắt ở static analysis, đắt nhất khi mất tiền trên mainnet. Read left→right as time, but security is a LOOP: the cost of a bug explodes the further right it is caught.

Threat modeling / Mô hình hóa mối đe dọa

Before writing code, write down / Trước khi viết mã, ghi ra:

- **Assets** — what is worth stealing? (funds, ownership, price feeds) *Tài sản đáng đánh cắp?*
- **Actors** — who can act? users, admins, other contracts, the proposer *Ai có thể hành động?*
- **Trust assumptions** — what do you assume is honest? make each one explicit *Giả định tin cậy?*
- **Invariants** — what must **always** hold? e.g. "sum of balances == totalSupply" *Bất biến luôn đúng?*

Invariants become your test oracles and your monitoring alerts. Write them first. /
Bất biến trở thành tiêu chí kiểm thử và cảnh báo giám sát.

Static analysis — Slither / Phân tích tĩnh

- **Slither** (Trail of Bits): analyzes Solidity **without running it**; seconds, no test setup
Slither phân tích Solidity mà không cần chạy; vài giây
- **Runs on Hardhat projects** directly: `slither .` auto-detects and compiles
Chạy thẳng trên dự án Hardhat: `slither .`
- Detects reentrancy, unchecked calls, tx.origin, uninitialized storage, and ~90 more
Phát hiện reentrancy, call không kiểm tra, tx.origin, ...

```
pipx install slither-analyzer    # or pip install --user slither-analyzer
slither .                        # put it in CI on every commit
```

Cheapest possible bug-catch: a linter that knows attack patterns. Triage the findings — some are informational. / Cách bắt lỗi rẻ nhất; hãy phân loại kết quả — có cái chỉ là thông tin.

Dynamic testing — fuzz, property, invariant / Kiểm thử động

- **Property test**: assert a rule over **many inputs**, not one example
Kiểm thử thuộc tính: khẳng định một quy tắc trên nhiều đầu vào
- **Fuzzing**: throw random inputs to break an assertion; **invariant testing**: random *sequences* of calls, checking an invariant after each
Fuzz: đầu vào ngẫu nhiên; invariant: chuỗi lời gọi ngẫu nhiên
- In **Hardhat**: property-style loops / `fast-check`; alternatives **Echidna**, Foundry (no toolchain switch required)
Trong Hardhat: vòng lặp thuộc tính / fast-check; lựa chọn khác: Echidna, Foundry

Invariant example: after ANY sequence of deposit/withdraw,
`sum(balances) == address(this).balance`

Symbolic & formal methods / Ký hiệu & hình thức

- **Symbolic execution (Mythril)**: explores *all* paths, solving for inputs that reach a bad state
Thực thi ký hiệu (Mythril): duyệt mọi nhánh, giải ra đầu vào dẫn tới trạng thái xấu
- **Formal verification (Certora)**: *prove* the code satisfies a written spec — not just test it
Kiểm chứng hình thức (Certora): CHỨNG MINH mã thỏa đặc tả — không chỉ test
- Strongest guarantees, highest effort — used on the most valuable protocols
Đảm bảo mạnh nhất, tốn công nhất — dùng cho giao thức giá trị cao

Tests show the presence of bugs on the paths you tried; proofs show their **absence** on all paths in scope. / Test cho thấy có lỗi trên nhánh đã thử; chứng minh cho thấy KHÔNG lỗi trên mọi nhánh trong phạm vi.

Audits — what they are (and aren't) / Kiểm toán

- An **audit** = expert humans read your code against a threat model, time-boxed
Kiểm toán = chuyên gia đọc mã theo mô hình đe dọa, trong thời gian giới hạn
- It is a **scoped snapshot**: this commit, this scope, this deadline
Là ảnh chụp có phạm vi: đúng commit, đúng phạm vi, đúng hạn
- It is **not** a guarantee. Post-audit code changes, integrations, and market conditions void it
KHÔNG phải bảo chứng. Đổi mã sau kiểm toán, tích hợp mới, thị trường đều làm mất hiệu lực

"Audited ≠ safe." Many exploited protocols were audited. Audits reduce risk; they do not remove it. / Nhiều giao thức bị hack đã được kiểm toán. Kiểm toán giảm rủi ro, không xóa bỏ.

Bug bounties & monitoring / Săn lỗi & giám sát

- **Bug bounty:** pay white-hats to report bugs before black-hats exploit them
Bug bounty: trả tiền cho hacker mũ trắng báo lỗi trước khi mũ đen khai thác
- **Immunefi** is the dominant platform; top payouts reach **millions of USD**
Immunefi là nền tảng lớn nhất; thưởng cao tới hàng triệu USD
- **Monitoring:** watch on-chain events, alert on invariant breaks, keep an incident runbook + pause switch
Giám sát: theo dõi sự kiện, cảnh báo khi bất biến bị phá, có kịch bản ứng phó + nút tạm dừng

Security is a **loop**, not a finish line: every finding returns to the threat model. / An toàn là VÒNG LẶP, không phải vạch đích.

Bridge hacks dominate losses / Cầu nối thống trị thiệt hại

- **Cross-chain bridges** hold huge pooled balances and complex trust assumptions → prime targets (hook S11)

Cầu nối liên chuỗi giữ số dư gộp khổng lồ + giả định tin cậy phức tạp ⇒ mục tiêu hàng đầu

- **Ronin, 2022** — ~\$625M: attacker compromised validator keys (5 of 9) — a **key-management**, not Solidity, failure

Ronin 2022 ~625 triệu \$: chiếm khóa validator — lỗi quản lý khóa

- **Wormhole, 2022** — ~\$325M: a signature-verification flaw let the attacker mint unbacked tokens

Wormhole 2022 ~325 triệu \$: lỗi kiểm chữ ký ⇒ đúc token không có bảo chứng

The biggest losses are often **operational** (keys, signatures, upgrades), not a single clever Solidity trick. / Thiệt hại lớn nhất thường là lỗi vận hành, không phải một mẹo Solidity.

Secure-coding checklist / Danh mục viết mã an toàn

- Follow **CEI**; add a `nonReentrant` guard on value-moving functions *theo CEI + guard*
- **Explicit access control**; events on privileged actions; two-step ownership *phân quyền rõ ràng*
- Prefer **battle-tested libraries** (OpenZeppelin) over rolling your own *dùng thư viện đã kiểm chứng*
- **Validate inputs**: zero address, zero amount, array lengths *kiểm tra đầu vào*
- Check **every** external-call return; treat all external addresses as hostile *kiểm mọi giá trị trả về*
- No `tx.origin` auth; no on-chain randomness; multiply-before-divide *không tx.origin, không RNG on-chain*
- **Slither in CI**; write **invariants**; get an **audit**; run a **bounty**; **monitor** *tự động hóa & giám sát*

Ethics — this is a defensive discipline / Đạo đức

- Everything today is for **building safer systems** and **AUTHORIZED testing** only
Mọi thứ hôm nay là để dựng hệ thống an toàn hơn và kiểm thử ĐƯỢC PHÉP
- Exploit **only** contracts you own, on testnets or in a lab you control
Chỉ khai thác hợp đồng bạn sở hữu, trên testnet hoặc lab của bạn
- Attacking others' contracts is **theft and a crime** — legal consequences are real
Tấn công hợp đồng người khác là trộm cắp và phạm pháp
- Found a real bug? **Responsible disclosure** via a bug bounty (Immunefi), never self-help
Tìm thấy lỗi thật? Công bố có trách nhiệm qua bug bounty, đừng tự xử

White-hat = permission + disclosure. Same skills, opposite intent. / Mũ trắng = được phép + công bố. Cùng kỹ năng, ngược ý định.



Break 10' → Lab 8 / Giải lao

Exploit the bank, then make the exploit fail

Khai thác ngân hàng, rồi làm cuộc khai thác thất bại

Lab 8 — exploit then patch / Khai thác rồi vá

EtherBank holds 30 ETH from three honest depositors. You are Eve, with a 1 ETH stake. /
EtherBank giữ 30 ETH của ba người gửi. Bạn là Eve với stake 1 ETH.

Step	Task / Nhiệm vụ
8.1	Complete <code>Attacker</code> + a Chai test that drains the bank to 0 / rút cạn về 0
8.2	Patch <code>withdraw()</code> with CEI + guard ; the same attack now reverts / vá rồi tấn công bị revert
8.3	Run <code>slither</code> ; read the <code>reentrancy-eth</code> finding / chạy Slither
8.4	<i>(bonus)</i> find & fix the unprotected <code>setOwner</code> via a property test / lỗi kiểm soát truy cập

Runs on the **local Hardhat chain** (no network needed); also works on **TrustKeys** with a funded test key. / Chạy trên chain Hardhat cục bộ; cũng chạy trên TrustKeys.

Lab 8 — the exploit & the fix / Khai thác & bản vá

```
// 8.1 the drain (vulnerable bank)
const atk = await Attacker.deploy(bank.target, { value: oneEth });
await atk.attack(); await atk.collect();
expect(await provider.getBalance(bank.target)).to.equal(0n); // drained!
```

```
// 8.2 the fix (EtherBankSecure)
function withdraw() external nonReentrant {
    uint256 bal = balances[msg.sender];
    require(bal > 0);
    balances[msg.sender] = 0; // EFFECT first
    (bool ok,) = msg.sender.call{value: bal}(""); // INTERACTION last
    require(ok, "transfer failed");
}
```

If Slither won't install on a lab machine, read `lab/solutions/captured-slither-output.txt` . / Nếu không cài được Slither, đọc file kết quả đã lưu.

Milestone — Midterm MCQ next session / Mốc: trắc nghiệm giữa kỳ (S9)

- **Session 9** opens with the **60-minute midterm MCQ** covering **Sessions 1-8**
Buổi 9 mở đầu bằng bài trắc nghiệm 60 phút, phạm vi Buổi 1-8
- Closed-book, in-session; bring a charged laptop for the S9 lab afterwards
Không tài liệu, làm tại lớp; mang laptop cho phần lab S9 sau đó
- Review checklist below — and the exam-theme list in the instructor notes
Danh mục ôn tập bên dưới — và danh sách chủ đề thi trong ghi chú giảng viên

Weighting and format follow the syllabus §5 / Midterm document. Come with questions. / Trọng số & hình thức theo đề cương §5.

Midterm review — S1-S8 themes / Ôn tập giữa kỳ

Sessions	Exam themes / Chủ đề ôn
S1-S2	wallets, keys, addresses, tx lifecycle / ví, khóa, địa chỉ, vòng đời giao dịch
S3	hashing, Merkle proofs, ECDSA, nonce reuse, EIP-712 / băm, Merkle, chữ ký
S4	Bitcoin consensus, PoW, forks, finality / đồng thuận, PoW, fork
S5	EVM, gas, EIP-1559, accounts, MEV / EVM, gas, MEV
S6	Solidity: visibility, view/pure/payable, storage vs memory, custom errors / Solidity
S7	ERC-20 / 721 / 1155, approvals, metadata / các chuẩn token
S8	reentrancy + CEI, access control, oracle/flash-loan, Slither, "audited $\neq$ safe" / an toàn

Homework / Bài về nhà (trước Buổi 9)

1. **Finish Lab 8:** patched `EtherBank` with `npx hardhat test` all green (drain proven, then blocked). / hoàn thành Lab 8, test xanh.
2. Write **3 sentences** on why "audited $\neq$ safe", citing one real incident from today. / viết 3 câu về "kiểm toán $\neq$ an toàn".
3. *(optional)* Try one **Ethernaut** level (`Reentrancy` or `Fallback`) or a **Damn Vulnerable DeFi** challenge. / thử một màn Ethernaut / Damn Vulnerable DeFi.
4. **Review S1-S8** for the midterm MCQ — use the checklist above. / ôn Buổi 1–8 cho trắc nghiệm.

Next / Buổi sau

Session 9 — Full-stack DApp + Midterm MCQ

Buổi 9 — DApp Full-stack + Trắc nghiệm giữa kỳ

viem + wagmi + React; wallet UX; events & indexing; account abstraction (ERC-4337)

60-minute midterm MCQ first / Trắc nghiệm 60 phút mở đầu

References / Tài liệu tham khảo (1/2)

- **Solidity docs — Security Considerations:**
<https://docs.soliditylang.org/en/latest/security-considerations.html>
- **OpenZeppelin Contracts — security & ReentrancyGuard :**
<https://docs.openzeppelin.com/contracts/5.x/api/utils#ReentrancyGuard>
- **Slither** (Trail of Bits): <https://github.com/crytic/slither>
- **Trail of Bits — Building Secure Contracts:** <https://secure-contracts.com/>
- **SWC Registry:** <https://swcregistry.io/> · **SCSVS:** <https://github.com/securing/SCSVS>

References / Tài liệu tham khảo (2/2)

- **The DAO / hard fork** (background): <https://ethereum.org/en/history/> (DAO fork), Ethereum Classic: <https://ethereumclassic.org/>
- **Parity multisig freeze** post-mortems (Parity Technologies security alerts, 2017)
- **Ethernaut** (wargame): <https://ethernaut.openzeppelin.com/> · **Damn Vulnerable DeFi**: <https://www.damnvulnerabledefi.xyz/>
- **Immunefi** (bug bounties): <https://immunefi.com/> · **rekt.news** (incident write-ups): <https://rekt.news/>
- **Chainlink** (oracles / price feeds): <https://docs.chain.link/> · **Certora**: <https://www.certora.com/> · **Mythril**: <https://github.com/Consensys/mythril>

Loss figures (The DAO ~3.6M ETH; Parity ~514k ETH; Ronin ~\$625M; Wormhole ~\$325M) are widely reported approximate values as of early 2026. / Các con số thiệt hại là giá trị xấp xỉ được đưa tin rộng rãi.